

Notebook_ca3

November 7, 2025

1 IND320 Course Project, Part 3

1.1 Code access and direct links

- The project is deployed here: [ind320-henrikengdal-project](#)
- The code is accessible at the repository: [henrikengdal/ind320-henrikengdal-project](#)

1.2 AI Usage

AI plays a multifaceted role throughout this project, primarily serving as an assistant and analytical tool. The project leverages AI in several areas:

Development and Code Generation: AI assists in writing and optimizing code for the application.

Data Analysis and Insights: AI helps analyze data patterns and identifying trends. It assists in generating meaningful statistical summaries and suggesting appropriate visualization techniques for the given data.

Documentation and Communication: AI supports the creation of clear documentation, such as code comments, and user interface text. It helps structure the project documentation and ensures technical concepts are communicated effectively.

Problem-Solving and Debugging: Throughout the development process, AI serves as a coding companion, helping troubleshoot issues, optimize data processing workflows, and suggesting best practices.

1.3 Defining representatives

```
[ ]: import pandas as pd
price_areas = {
    'N01': ('Oslo', 10.7522, 59.9139),
    'N02': ('Kristiansand', 7.9956, 58.1467),
    'N03': ('Trondheim', 10.3951, 63.4305),
    'N04': ('Tromsø', 18.9553, 69.6496),
    'N05': ('Bergen', 5.3221, 60.3913)
}
df = pd.DataFrame.from_dict(price_areas, orient='index', columns=['City', 'Longitude', 'Latitude'])
```

1.4 API Download

```
[44]: import requests_cache
from datetime import datetime

# Fallback if retry_requests is not available
try:
    from retry_requests import retry
except Exception:
    def retry(session, retries: int = 0, backoff_factor: float = 0.0):
        return session

# Use ERA5 archive endpoint for historical data like 2019
BASE_URL = "https://archive-api.open-meteo.com/v1/era5"
HOURL_VARS = [
    "temperature_2m",
    "precipitation",
    "windspeed_10m",
    "windgusts_10m",
    "winddirection_10m",
]

# Cached session with retries for resilient API calls
_session = requests_cache.CachedSession(
    cache_name="openmeteo_cache", backend="sqlite", expire_after=60 * 60 * 24
)
_session = retry(_session, retries=3, backoff_factor=0.2)

def download_weather_data(longitude: float, latitude: float, year: int,
    ↪session=_session) -> pd.DataFrame:
    """Download hourly ERA5 weather for a given location and year.

    Parameters
    - longitude, latitude: geographic coordinates
    - year: e.g. 2019

    Returns
    - DataFrame with columns: time + requested hourly variables
    """
    start_date = f"{year}-01-01"
    end_date = f"{year}-12-31"

    params = {
        "latitude": latitude,
        "longitude": longitude,
        "start_date": start_date,
        "end_date": end_date,
```

```

        "hourly": ",".join(HOUR_VARS),
        "timezone": "Europe/Oslo",
    }

    r = session.get(BASE_URL, params=params, timeout=30)
    r.raise_for_status()
    data = r.json()

    if "hourly" not in data:
        raise ValueError(f"Unexpected API response. Top-level keys: {list(data.keys())}")

    hourly = data["hourly"]
    # Build DataFrame in a robust way, filling missing variables with NA
    df = pd.DataFrame({"time": pd.to_datetime(hourly["time"])})
    for var in HOUR_VARS:
        df[var] = hourly.get(var)
    return df

# Example: Bergen 2019
df_bergen = download_weather_data(price_areas['N05'][1], price_areas['N05'][2],
    ↪2019)
df_bergen.head()

```

```

[44]:
      time  temperature_2m  precipitation  windspeed_10m \
0 2019-01-01 00:00:00         5.7          0.7         37.0
1 2019-01-01 01:00:00         5.8          0.2         41.0
2 2019-01-01 02:00:00         6.1          0.7         42.0
3 2019-01-01 03:00:00         6.3          0.5         40.9
4 2019-01-01 04:00:00         5.8          1.1         41.2

      windgusts_10m  winddirection_10m
0          99.7          263
1          107.3          278
2          112.0          286
3          105.8          298
4          110.2          315

```

1.5 Outliers and anomalies

```

[45]: import numpy as np
      from sklearn.neighbors import LocalOutlierFactor
      from scipy.fft import dct, idct

      import plotly.graph_objects as go

      def plot_temperature_with_outliers(df, dct_cutoff=200, n_std=3.5):

```

```

"""
Plot temperature with SPC outlier detection using DCT high-pass filtering.

Parameters:
- df: DataFrame with 'time' and 'temperature_2m' columns
- dct_cutoff: Frequency cutoff for DCT high-pass filter
- n_std: Number of standard deviations for outlier bounds

Returns:
- fig: Plotly figure
- summary: Dictionary with outlier statistics
"""
temp_data = df['temperature_2m'].values

# Apply DCT high-pass filtering for SATV
temp_dct = dct(temp_data, type=2, norm='ortho')
temp_dct[:dct_cutoff] = 0 # Zero out low frequencies
satv = idct(temp_dct, type=2, norm='ortho')

# Robust statistics for outlier detection
median_satv = np.median(satv)
mad_scale = np.median(np.abs(satv - median_satv)) * 1.4826 # MAD scale
↪ estimator

lower_bound = median_satv - n_std * mad_scale
upper_bound = median_satv + n_std * mad_scale

# Identify outliers
outliers = (satv < lower_bound) | (satv > upper_bound)
outlier_indices = np.where(outliers)[0]

# Create plot
fig = go.Figure()

# Temperature line
fig.add_trace(go.Scatter(
    x=df['time'],
    y=temp_data,
    mode='lines',
    name='Temperature',
    line=dict(color='#1f77b4')
))

# Control bounds
fig.add_trace(go.Scatter(
    x=df['time'],
    y=[lower_bound] * len(df),

```

```

        mode='lines',
        name='Lower Bound',
        line=dict(color='red', dash='dash')
    ))

    fig.add_trace(go.Scatter(
        x=df['time'],
        y=[upper_bound] * len(df),
        mode='lines',
        name='Upper Bound',
        line=dict(color='red', dash='dash')
    ))

    # Outliers
    if len(outlier_indices) > 0:
        fig.add_trace(go.Scatter(
            x=df['time'].iloc[outlier_indices],
            y=temp_data[outlier_indices],
            mode='markers',
            name='Outliers',
            marker=dict(color='orange', size=6)
        ))

    fig.update_layout(
        title='Temperature with Robust SPC Outlier Highlight',
        legend=dict(orientation='h')
    )

    # Summary statistics
    summary = {
        'median_satv': median_satv,
        'mad_scale': mad_scale,
        'lower_bound': lower_bound,
        'upper_bound': upper_bound,
        'outlier_count': len(outlier_indices),
        'pct_outliers': (len(outlier_indices) / len(temp_data)) * 100,
        'dct_cutoff': dct_cutoff,
        'n_std': n_std
    }

    return fig, summary

def plot_precipitation_with_anomalies(df, contamination=0.01, n_neighbors=20):
    """
    Plot precipitation with LOF anomaly detection.

    Parameters:
    """

```

```

- df: DataFrame with 'time' and 'precipitation' columns
- contamination: Proportion of outliers expected
- n_neighbors: Number of neighbors for LOF

Returns:
- fig: Plotly figure
- summary: Dictionary with anomaly statistics
"""
precip_data = df['precipitation'].values.reshape(-1, 1)

# LOF anomaly detection
lof = LocalOutlierFactor(n_neighbors=n_neighbors,
contamination=contamination)
anomaly_labels = lof.fit_predict(precip_data)

# Get anomalies
anomaly_mask = anomaly_labels == -1
anomaly_indices = np.where(anomaly_mask)[0]

# Create plot
fig = go.Figure()

# Precipitation line
fig.add_trace(go.Scatter(
    x=df['time'],
    y=df['precipitation'],
    mode='lines',
    name='Precipitation',
    line=dict(color='#1f77b4')
))

# Anomalies
if len(anomaly_indices) > 0:
    fig.add_trace(go.Scatter(
        x=df['time'].iloc[anomaly_indices],
        y=df['precipitation'].iloc[anomaly_indices],
        mode='markers',
        name='Anomalies',
        marker=dict(color='crimson', size=6)
    ))

fig.update_layout(
    title='Precipitation with LOF Anomalies',
    legend=dict(orientation='h')
)

# Summary statistics

```

```

summary = {
    'n_anomalies': len(anomaly_indices),
    'pct_anomalies': (len(anomaly_indices) / len(precip_data)) * 100,
    'contamination': contamination,
    'n_neighbors': n_neighbors
}

return fig, summary

# Test the functions with Bergen data
fig_temp, summary_temp = plot_temperature_with_outliers(df_bergen)
fig_precip, summary_precip = plot_precipitation_with_anomalies(df_bergen)

print("Temperature outlier summary:", summary_temp)
print("Precipitation anomaly summary:", summary_precip)

fig_temp.show()
fig_precip.show()

```

```

Temperature outlier summary: {'median_satv': -0.11082683040555485, 'mad_scale':
1.9377283075333809, 'lower_bound': -6.892875906772389, 'upper_bound':
6.67122245961278, 'outlier_count': 51, 'pct_outliers': 0.5821917808219178,
'dct_cutoff': 200, 'n_std': 3.5}
Precipitation anomaly summary: {'n_anomalies': 80, 'pct_anomalies':
0.91324200913242, 'contamination': 0.01, 'n_neighbors': 20}

/Users/henrikengdal/Documents/GitHub/IND320-henrikengdal-
project/.conda/lib/python3.11/site-packages/sklearn/neighbors/_lof.py:322:
UserWarning:

```

Duplicate values are leading to incorrect results. Increase the number of neighbors for more accurate results.

1.6 Seasonal-Trend decomposition using LOESS (STL)

```

[46]: from statsmodels.tsa.seasonal import STL
from plotly.subplots import make_subplots
import pandas as pd
import plotly.graph_objects as go

def plot_stl_decomposition(df, value_col='production_mwh', time_col='time',
                           period=24*7, seasonal=7, trend=None, robust=True,
                           title_prefix="STL Decomposition"):
    """
    Perform STL decomposition on time series data and create an interactive
    plot.
    """

```

```

Parameters:
- df: DataFrame with time series data
- value_col: Column name containing the values to decompose
- time_col: Column name containing datetime values
- period: Seasonal period (must be an integer >= 2). For hourly data, use
↳ 24 for daily seasonality.
- seasonal: Length of seasonal smoother (odd integer). If even, it's
↳ incremented to next odd.
- trend: Length of trend smoother (odd integer) or None for auto. If even,
↳ it's incremented to next odd.
- robust: Whether to use robust fitting (default: True)
- title_prefix: Prefix for plot title

Returns:
- fig: Plotly figure with STL decomposition
- stl_result: STL decomposition result object
"""

# Set time as index if not already
if time_col in df.columns:
    ts_data = df.set_index(time_col)[value_col]
else:
    ts_data = df[value_col]

# Remove any NaN values
ts_data = ts_data.dropna()

# Coerce and validate STL parameters
period_int = int(round(period))
if period_int < 2:
    raise ValueError(f"period must be >= 2, got {period}")

def _ensure_odd(n):
    if n is None:
        return None
    n_int = int(round(n))
    return n_int if n_int % 2 == 1 else n_int + 1

seasonal_odd = _ensure_odd(seasonal)
trend_odd = _ensure_odd(trend)

# Perform STL decomposition
stl = STL(ts_data, period=period_int, seasonal=seasonal_odd,
↳ trend=trend_odd, robust=robust)
stl_result = stl.fit()

# Create subplots

```



```

fig = make_subplots(
    rows=4, cols=1,
    subplot_titles=['Original', 'Trend', 'Seasonal', 'Residual'],
    shared_xaxes=True,
    vertical_spacing=0.08
)

# Original data
fig.add_trace(
    go.Scatter(x=stl_result.observed.index, y=stl_result.observed.values,
               mode='lines', name='Original', line=dict(color='blue')),
    row=1, col=1
)

# Trend component
fig.add_trace(
    go.Scatter(x=stl_result.trend.index, y=stl_result.trend.values,
               mode='lines', name='Trend', line=dict(color='red')),
    row=2, col=1
)

# Seasonal component
fig.add_trace(
    go.Scatter(x=stl_result.seasonal.index, y=stl_result.seasonal.values,
               mode='lines', name='Seasonal', line=dict(color='green')),
    row=3, col=1
)

# Residual component
fig.add_trace(
    go.Scatter(x=stl_result.resid.index, y=stl_result.resid.values,
               mode='lines', name='Residual', line=dict(color='orange')),
    row=4, col=1
)

# Update layout
fig.update_layout(
    title=f"{title_prefix} - Period: {period_int}, Seasonal: {
↪{seasonal_odd}, Robust: {robust}",
    height=800,
    showlegend=False
)

# Update y-axis labels
fig.update_yaxes(title_text=value_col.replace('_', ' ').title(), row=1,
↪col=1)
fig.update_yaxes(title_text="Trend", row=2, col=1)

```

```

fig.update_yaxes(title_text="Seasonal", row=3, col=1)
fig.update_yaxes(title_text="Residual", row=4, col=1)

return fig, stl_result

# Test the function with Bergen temperature data (use daily seasonality)
fig_stl_temp, stl_temp_result = plot_stl_decomposition(
    df_bergen,
    value_col='temperature_2m',
    time_col='time',
    period=24,          # Daily cycle for hourly data
    seasonal=13,        # Seasonal smoother length (odd)
    title_prefix="Temperature STL Decomposition (Bergen 2019)"
)

fig_stl_temp.show()

# Display decomposition statistics
print("STL Decomposition Results:")
print(f"Seasonal strength: {1 - stl_temp_result.resid.var() / (stl_temp_result.
    ↪seasonal + stl_temp_result.resid).var():.3f}")
print(f"Trend strength: {1 - stl_temp_result.resid.var() / (stl_temp_result.
    ↪trend + stl_temp_result.resid).var():.3f}")

```

STL Decomposition Results:
Seasonal strength: 0.670
Trend strength: 0.951

1.7 Spectrogram

```

[47]: import numpy as np
from plotly.subplots import make_subplots
from scipy import signal
import pandas as pd

import plotly.graph_objects as go

def create_spectrogram(df, value_col='production_mwh', time_col='time',
    window='hann', nperseg=256, noverlap=None,
    title_prefix="Spectrogram"):
    """
    Create a spectrogram visualization for time series data.

    Parameters:
    - df: DataFrame with time series data
    - value_col: Column name containing the values to analyze
    - time_col: Column name containing datetime values
    
```

```

- window: Window function ('hann', 'hamming', 'blackman', etc.)
- nperseg: Length of each segment (window size)
- noverlap: Number of points to overlap between segments (default: nperseg//
↪8)
- title_prefix: Prefix for plot title

Returns:
- fig: Plotly figure with spectrogram
- spec_data: Dictionary with spectrogram data (frequencies, times, Sxx)
"""

# Set default overlap if not provided
if noverlap is None:
    noverlap = nperseg // 8

# Extract time series data
if time_col in df.columns:
    ts_data = df.set_index(time_col)[value_col]
else:
    ts_data = df[value_col]

# Remove NaN values
ts_data = ts_data.dropna()

# Calculate sampling frequency (assuming regular intervals)
time_diff = ts_data.index.to_series().diff().median()
fs = 1 / (time_diff.total_seconds() / 3600) # Convert to samples per hour

# Compute spectrogram
frequencies, times, Sxx = signal.spectrogram(
    ts_data.values,
    fs=fs,
    window=window,
    nperseg=nperseg,
    noverlap=noverlap,
    scaling='density'
)

# Convert power spectral density to dB
Sxx_db = 10 * np.log10(Sxx + 1e-10) # Add small value to avoid log(0)

# Convert time segments back to datetime
start_time = ts_data.index[0]
time_segments = [start_time + pd.Timedelta(hours=t) for t in times]

# Create spectrogram plot
fig = go.Figure(data=go.Heatmap(

```

```

        x=time_segments,
        y=frequencies,
        z=Sxx_db,
        colorscale='Viridis',
        colorbar=dict(title="Power Spectral Density (dB)")
    ))

fig.update_layout(
    title=f"{title_prefix} - Window: {window}, Segment length: {nperseg}",
    xaxis_title="Time",
    yaxis_title="Frequency (cycles/hour)",
    height=600
)

# Prepare return data
spec_data = {
    'frequencies': frequencies,
    'times': times,
    'time_segments': time_segments,
    'Sxx': Sxx,
    'Sxx_db': Sxx_db,
    'sampling_freq': fs,
    'window': window,
    'nperseg': nperseg,
    'noverlap': noverlap
}

return fig, spec_data

# Test the function with Bergen temperature data
fig_spec, spec_data = create_spectrogram(
    df_bergen,
    value_col='temperature_2m',
    time_col='time',
    window='hann',
    nperseg=512, # Larger window for better frequency resolution
    noverlap=256, # 50% overlap
    title_prefix="Temperature Spectrogram (Bergen 2019)"
)

fig_spec.show()

# Display spectrogram statistics
print("Spectrogram Analysis Results:")
print(f"Sampling frequency: {spec_data['sampling_freq']:.2f} samples/hour")
print(f"Frequency resolution: {spec_data['frequencies'][1] - spec_data['frequencies'][0]:.4f} cycles/hour")

```

```
print(f"Time resolution: {spec_data['times'][1] - spec_data['times'][0]:.2f}␣
↪hours")
print(f"Number of frequency bins: {len(spec_data['frequencies'])}")
print(f"Number of time segments: {len(spec_data['times'])}")
print(f"Power range: {spec_data['Sxx_db'].min():.1f} to {spec_data['Sxx_db']␣
↪max():.1f} dB")
```

Spectrogram Analysis Results:

Sampling frequency: 1.00 samples/hour

Frequency resolution: 0.0020 cycles/hour

Time resolution: 256.00 hours

Number of frequency bins: 257

Number of time segments: 33

Power range: -48.8 to 36.6 dB

1.8 Word log

For this part, I found myself using AI more frequently to understand what was being asked for in the CA description than I did with the previous two parts. I think this was mainly because the expectations were phrased in a way that required some interpretation, and AI tools like ChatGPT helped me clarify the intent before I started implementing anything. Either way, I think we’ve arrived at something that covers all the necessary points and demonstrates a clear understanding of the task requirements.

Starting with the Streamlit experience, this section was relatively straightforward, but it did include a few challenges that required trial and error. The most difficult part was understanding how to reorder and manage the pages correctly while maintaining consistent naming and navigation. Once I figured that out, the implementation process went much more smoothly. The “new A” page was implemented without major issues. I used GitHub Copilot to get a sense of what the “necessary UI elements” might look like, and it provided good suggestions that aligned closely with what I needed. There were still a few moments where I had to manually adjust the layout or functionality, but overall the experience was productive.

The “new B” page, however, brought a few challenges, especially with implementing caching for selections made in the “Energy Production Analysis” page. It took some testing and revisions to get everything working as intended, but I’m satisfied with the final behavior.

The notebook part of the assignment was also quite manageable. Since this section required creating and refining several methods, I used AI support mainly for generating documentation and improving code readability. This not only saved time but also helped maintain consistency across the different components of the project.

Overall, I gained a better understanding of how to structure the Streamlit application effectively.